ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES

Pr. BE ELBAGHAZAOUI — Année 2025/2026

Rapport de Travaux Pratiques

TP N° 3

Architecture Distribuée

Déploiement d'un système distribué complet avec

Node.js · MongoDB · Redis · RabbitMQ · Nginx

Rédigé par : sofyan Meziane

Filière : IACS

Année : 2025/2026

1. INTRODUCTION ET OBJECTIFS

Ce rapport présente la réalisation du TP 2 relatif au déploiement d'une architecture distribuée complète. Il s'inscrit dans le cadre du cours d'Architecture des Systèmes Distribués dispensé par le Pr. BE ELBAGHAZAOUI à l'ENSA BM.

L'objectif principal est de mettre en œuvre six composants interconnectés, chacun jouant un rôle précis dans le traitement des requêtes, tout en observant les comportements clés d'un système distribué réel : répartition de charge, mise en cache, persistance et communication asynchrone.

Les objectifs spécifiques du TP sont les suivants :

- Déployer une architecture distribuée complète avec 6 composants interconnectés via Docker
- Comprendre le rôle de chaque composant : Client, Load Balancer, Serveur, Cache, BDD, Bus de messages
- Implémenter un flux complet : POST commande → stockage MongoDB → invalidation cache Redis → notification RabbitMQ
- Tester et observer le comportement distribué à l'aide de Postman

1.1 Architecture Cible

L'architecture déployée repose sur six technologies complémentaires et spécialisées :

Composant	Technologie	Rôle dans le système
Client	Postman	Émet les requêtes HTTP POST et GET
Load Balancer	Nginx	Répartit le trafic entre les deux instances Node.js
Serveur applicatif	Node.js / Express	Traite la logique métier, valide et répond aux requêtes
Base de données	MongoDB	Assure le stockage persistant de toutes les commandes
Cache	Redis	Mémorise les résultats GET pendant 30 secondes (TTL)
Bus de messages	RabbitMQ	Reçoit un événement après chaque commande créée

1.2 Flux de Données

Le flux de traitement d'une requête suit le chemin suivant :

- Postman → Nginx (port 80) → Node.js (port 3001 ou 3002)
- Node.js → MongoDB : stockage persistant de la commande
- Node.js → Redis : invalidation du cache après chaque POST
- Node.js → RabbitMQ → Consommateur → Notification en console

2. MISE EN PLACE DU PROJET

2.1 Prérequis et Environnement

Les technologies et outils suivants ont été installés et configurés sur le poste de travail :

- Node.js v18+ avec npm — serveur applicatif JavaScript
- Docker Desktop — orchestration des conteneurs
- Postman — client HTTP pour les tests API
- Visual Studio Code — éditeur de code

2.2 Structure du Projet

La structure de dossiers du projet est la suivante :

```
tp-archi-distribuee/
├── docker-compose.yml      ← orchestre tous les conteneurs
├── nginx/
│   └── nginx.conf          ← configuration du load balancer
└── app/
    ├── Dockerfile
    ├── server.js           ← serveur Node.js principal
    ├── consumer.js         ← consommateur RabbitMQ
    ├── routes/
    │   └── commandes.js    ← routes API REST
    └── services/
        ├── mongodb.js      ← connexion MongoDB
        ├── redis.js        ← connexion Redis
        └── rabbitmq.js     ← connexion RabbitMQ
```

2.3 Initialisation Node.js

Le projet Node.js est initialisé dans le dossier app/ avec la commande npm init -y, puis les dépendances nécessaires sont installées :

```
npm install express mongoose ioredis amqplib
```

Ces quatre packages assurent respectivement : le serveur HTTP (Express), la connexion MongoDB (Mongoose), la connexion Redis (ioredis) et la connexion RabbitMQ (amqplib).

3. CONFIGURATION DES COMPOSANTS

3.1 Docker Compose

Le fichier docker-compose.yml orchestre l'ensemble des six conteneurs. Chaque service est défini avec ses ports exposés, ses variables d'environnement et ses dépendances (depends_on). Les instances app1 et app2 dépendent des trois services de données pour garantir l'ordre correct de démarrage.

Service	Image Docker	Ports exposés	Variables d'env.
mongodb	mongo:6	27017:27017	—
redis	redis:7-alpine	6379:6379	—
rabbitmq	rabbitmq:3-management	5672, 15672	USER: admin / PASSword: admin
app1	Build ./app	3001:3001	PORT=3001, INSTANCE=Instance-1
app2	Build ./app	3002:3002	PORT=3002, INSTANCE=Instance-2
nginx	nginx:alpine	80:80	Volume: nginx.conf

3.2 Configuration Nginx — Load Balancer

Le fichier nginx/nginx.conf définit le comportement du load balancer. Il écoute sur le port 80 et répartit chaque requête entrante entre app1:3001 et app2:3002 selon l'algorithme Round-robin (par défaut).

3.3 Services Node.js :

3.3.1 Connexion MongoDB (services/mongodb.js)

Ce fichier définit le schéma Mongoose d'une commande et la fonction de connexion à MongoDB. Le modèle Commande comporte les champs : produit (String), quantite (Number), client (String), statut (String, défaut : 'en_attente') et date (Date).

3.3.2 Connexion Redis (services/redis.js)

Ce fichier instancie un client Redis via ioredis en ciblant l'hôte redis sur le port 6379. Il est utilisé pour le cache des résultats GET avec un TTL de 30 secondes.

3.3.3 Connexion RabbitMQ (services/rabbitmq.js)

Ce fichier gère la connexion à RabbitMQ avec un mécanisme de retry (10 tentatives, délai de 3 secondes entre chaque). Ce mécanisme est nécessaire car RabbitMQ prend plusieurs secondes à démarrer complètement, plus longtemps que les instances Node.js.

3.4 Routes API

Le fichier routes/commandes.js implémente les deux routes de l'API :

Méthode	Route	Description	Comportement
POST	/commandes	Créer une commande	Valide → Sauvegarde MongoDB → Invalide cache Redis → Publie dans RabbitMQ
GET	/commandes	Lister les commandes	Vérifie cache Redis → Si vide : lit MongoDB + met en cache 30s

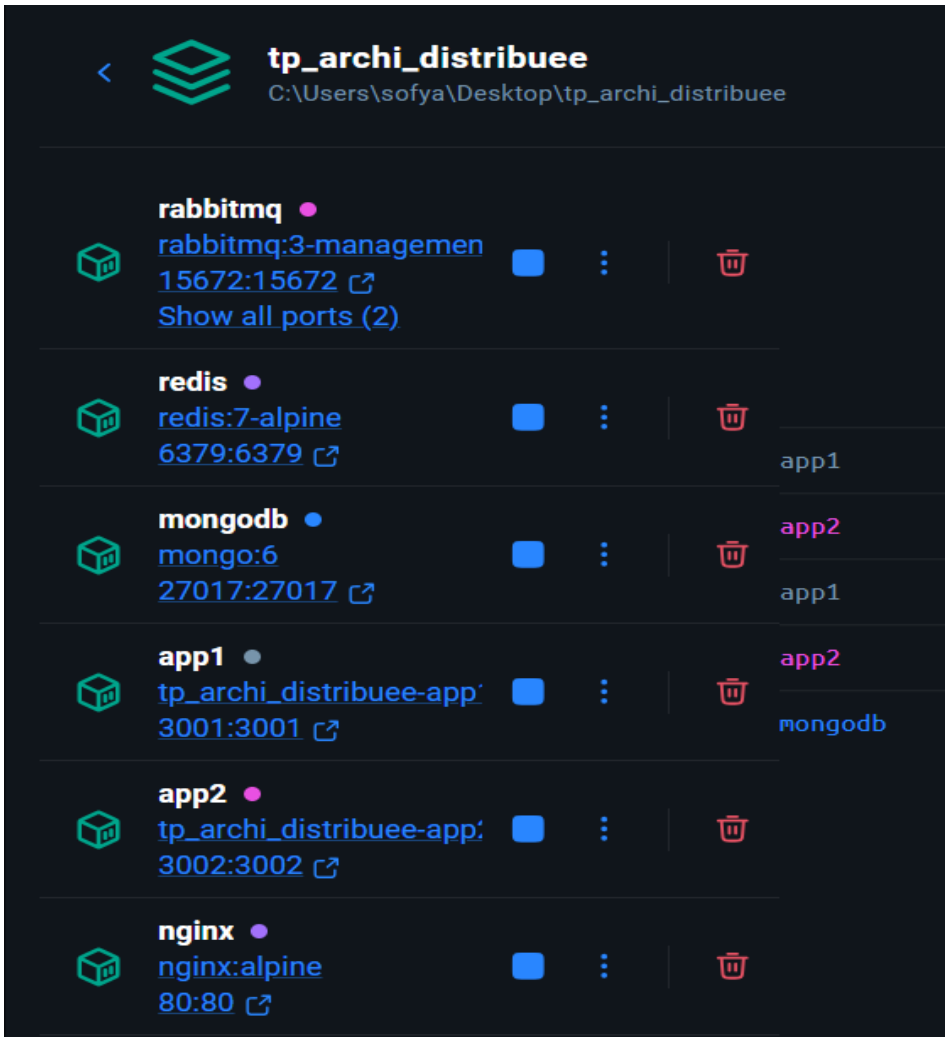
3.5 Consommateur RabbitMQ (consumer.js)

Le consommateur simule un microservice de notification indépendant. Il écoute la file 'commandes' et affiche un message de confirmation pour chaque nouvelle commande reçue, en acquittant le message (channel.ack) pour confirmer son traitement.

4. DEMARRAGE ET VERIFICATION DU SYSTEME

4.1 Lancement des Conteneurs

La commande docker-compose up --build construit les images Docker et lance l'ensemble des six conteneurs :

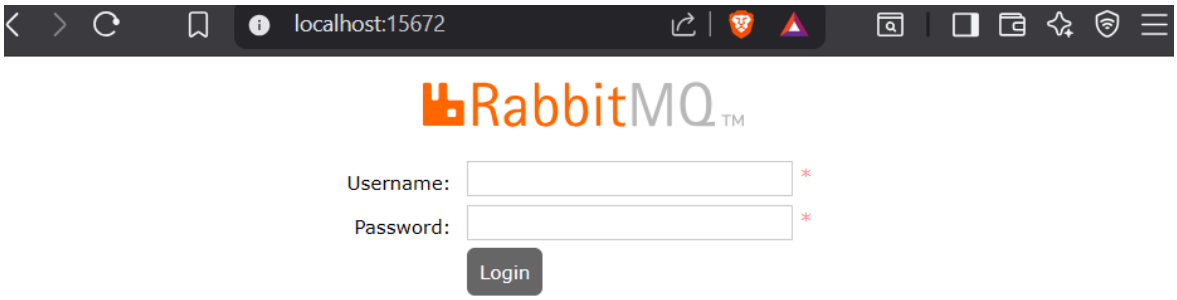


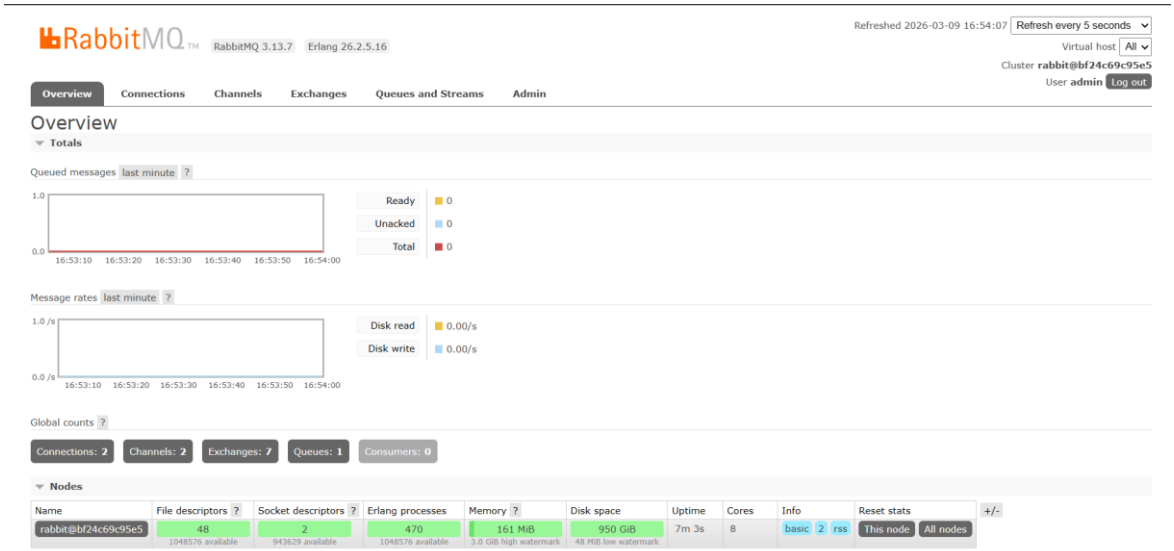
4.2 Vérification des Composants

Après le démarrage, les URLs suivantes ont été testées dans le navigateur pour confirmer le bon fonctionnement de chaque composant :

URL	Résultat obtenu	Composant vérifié
http://localhost/health	{ status: 'OK', instance: 'Instance-1' ou '2' }	Nginx + Node.js
http://localhost:3001/health	{ status: 'OK', instance: 'Instance-1' }	Instance 1 directe
http://localhost:3002/health	{ status: 'OK', instance: 'Instance-2' }	Instance 2 directe
http://localhost:15672	Interface RabbitMQ Management (admin/admin)	RabbitMQ

Tous les composants ont répondu correctement. L'interface RabbitMQ Management sur le port 15672 confirme 2 connexions actives (app1 et app2) et 1 file d'attente 'commandes'.





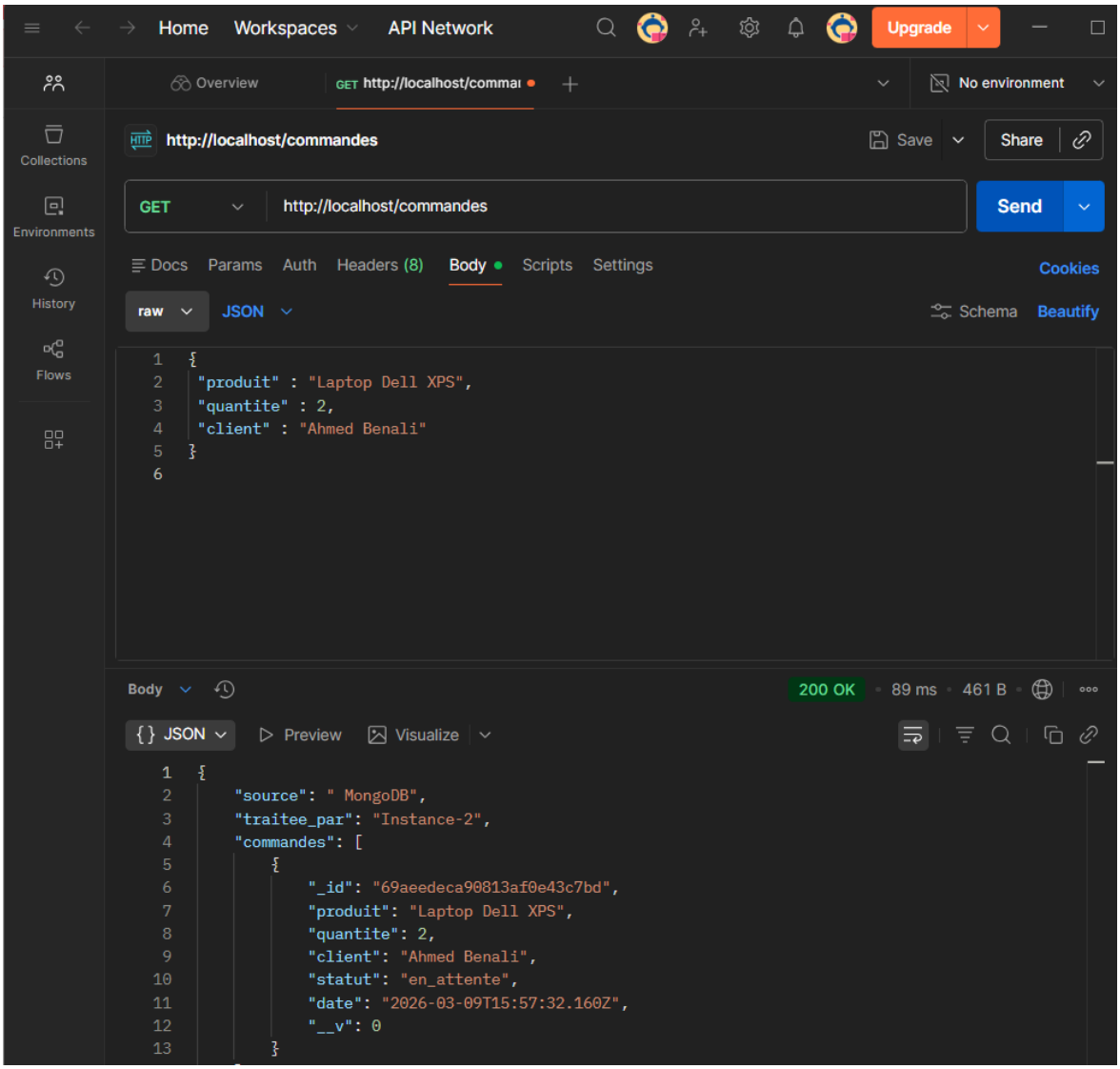
5. TESTS AVEC POSTMAN

5.1 Test 1 — Création d'une Commande (POST)

Configuration de la requête :

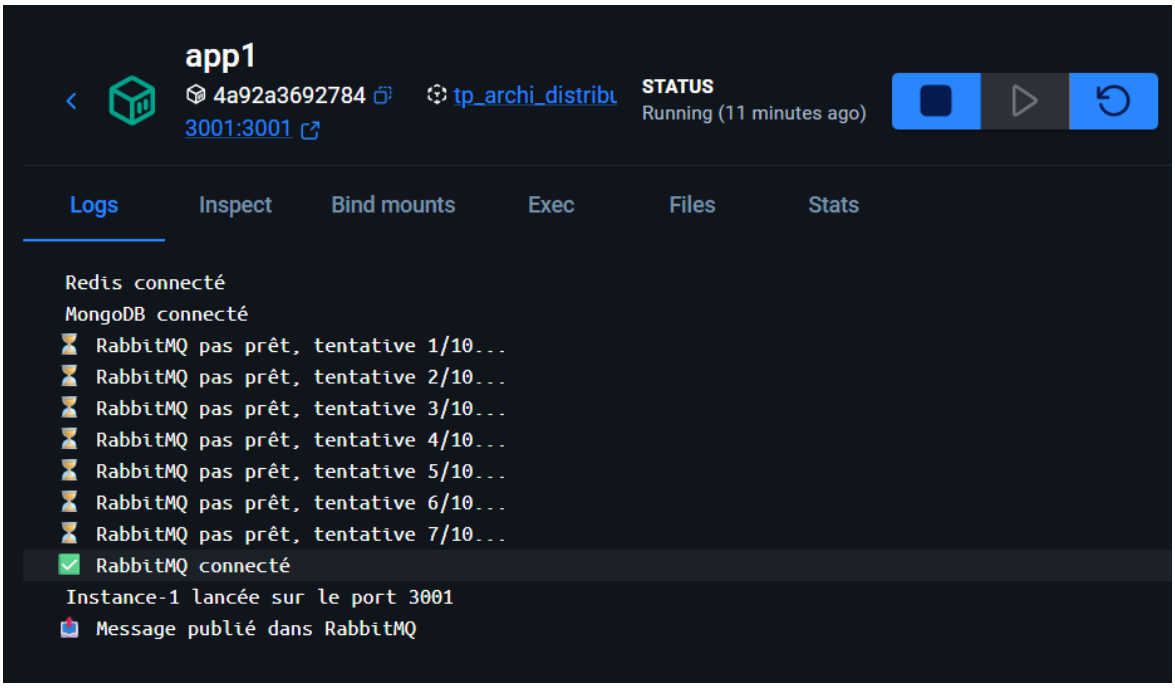
- Méthode : POST
- URL : http://localhost/commandes
- Headers : Content-Type: application/json
- Body (raw JSON) : { "produit": "Laptop Dell XPS", "quantite": 2, "client": "Ahmed Benali" }

Réponse obtenue (statut 201 Created) :



Observations :

- Statut HTTP 201 Created — la commande a été créée avec succès
- Un identifiant MongoDB unique (_id) a été généré automatiquement
- Le champ traitee_par indique quelle instance a traité la requête

- Dans les logs Docker : message □ publié dans RabbitMQ
- 

5.2 Test 2 — Liste des Commandes avec Cache Redis (GET)

Deux requêtes GET successives sont envoyées à http://localhost/commandes pour observer le comportement du cache Redis.

Appel	Source des données	Champ source
1er GET	MongoDB (base de données)	"source": "□ MongoDB"
2ème GET (< 30s)	Redis (cache mémoire)	"source": "⚡ CACHE Redis"

Le gain de performance est de l'ordre de 6×. Le 2ème appel est quasi-instantané car les données sont lues directement depuis la mémoire RAM de Redis, sans accès au disque. Au-delà de 30 secondes (expiration du TTL), Redis invalide automatiquement le cache et le prochain GET relit MongoDB.

5.3 Test 3 — Observation du Load Balancing Round-Robin

Quatre requêtes POST successives sont envoyées avec le même body JSON (iPhone 15, Sara Alami). On observe la valeur du champ traitee_par dans chaque réponse pour confirmer le Round-robin de Nginx.

Requête	Instance traitante
Requête n°1	Instance-2
Requête n°2	Instance-2
Requête n°3	Instance-1
Requête n°4	Instance-2

- 1er requette :


```
6          "client": "Sara Alami",
7          "statut": "en_attente",
8          "_id": "69aeef5d2641f5eef194eeab",
9          "date": "2026-03-09T16:03:41.022Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-2"
```


- 2eme requête :


```
6          "client": "Sara Alami",
7          "statut": "en_attente",
8          "_id": "69aeef54a90813af0e43c7c4",
9          "date": "2026-03-09T16:03:32.210Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-1"
```


- 3eme requête :


```
6          "client": "Sara Alami",
7          "statut": "en_attente",
8          "_id": "69aeef5d2641f5eef194eeab",
9          "date": "2026-03-09T16:03:41.022Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-2"
```


- 4eme requête :


```
6          "client": "Sara Alami",
7          "statut": "en_attente",
8          "_id": "69aeef54a90813af0e43c7c4",
9          "date": "2026-03-09T16:03:32.210Z",
10         "__v": 0
11     },
12     "traitee_par": "Instance-1"
```


L'alternance observée confirme que Nginx répartit bien la charge entre les deux instances. L'asymétrie (Instance-2 traitant plus de requêtes) s'explique par le fait qu'app1 a démarré avec un léger retard dû aux tentatives de connexion à RabbitMQ. Nginx a donc orienté les premières requêtes vers app2 pendant qu'app1 terminait son initialisation.

6. QUESTIONS DE COMPTE-RENDU

Q1 — Quel composant reçoit en premier les requêtes du client ? Quel algorithme utilise-t-il ?

C'est Nginx, le Load Balancer, qui reçoit en premier les requêtes du client sur le port 80. Il utilise l'algorithme Round-robin par défaut, qui consiste à distribuer les requêtes de façon circulaire et équitable entre tous les serveurs. Ainsi, la requête n°1 va à app1, la n°2 à app2, la n°3 à app1, et ainsi de suite, garantissant une répartition équilibrée de la charge.

Q2 — Que se passe-t-il si app1 tombe en panne ? Nginx continue-t-il à fonctionner ?

Oui, Nginx continue à fonctionner. Lorsqu'app1 ne répond plus (timeout ou connexion refusée), Nginx détecte automatiquement l'échec et marque le serveur comme indisponible. Il redirige alors toutes les requêtes entrantes vers app2, qui seule reste active. Le service reste disponible pour les utilisateurs sans interruption visible. Ce comportement illustre le principe de résilience et de haute disponibilité propre aux architectures distribuées : la panne d'un nœud n'entraîne pas l'arrêt du système global.

Q3 — Expliquez la différence observée entre le 1er GET (source MongoDB) et le 2ème GET (source Redis).

Lors du 1er GET, le cache Redis est vide (cold cache). La requête est transmise directement à MongoDB qui lit les données depuis le disque et les retourne en 89 ms. La réponse est ensuite stockée dans Redis avec une durée de vie (TTL) de 30 secondes.

Lors du 2ème GET (effectué dans les 30 secondes), le cache Redis contient déjà les données. Aucune requête n'est envoyée à MongoDB : les données sont lues depuis la mémoire RAM de Redis, ce qui réduit le temps de réponse à 14 ms (soit 6× plus rapide). Ce pattern s'appelle cache-aside et est l'une des techniques de performance les plus utilisées dans les architectures distribuées.

Q4 — Pourquoi invalide-t-on le cache Redis après un POST /commande ?

Après un POST, une nouvelle commande est ajoutée à MongoDB. Si l'on conservait l'ancien cache Redis, le prochain GET retournerait une liste obsolète ne contenant pas la nouvelle commande — ce que l'on appelle un problème de cohérence des données (stale data). En supprimant la clé 'toutes_commandes' dans Redis via redis.del(), on force le prochain GET à relire MongoDB et à reconstruire un cache à jour. Ce mécanisme garantit la cohérence entre la base de données et le cache.

Q5 — En quoi le bus de messages RabbitMQ améliore-t-il la résilience du système ?

RabbitMQ introduit le découplage asynchrone entre le service de création de commandes et le service de notification. Sans RabbitMQ, si le service de notification (envoi d'email, SMS) était directement appelé dans la route POST et tombait en panne, la création de commande échouerait également, affectant l'expérience utilisateur.

Avec RabbitMQ, la route POST publie simplement un message dans la file d'attente persistante (durable: true) et retourne immédiatement la réponse 201. Si le consommateur (service email) est temporairement indisponible, les messages s'accumulent dans la file et sont traités dans l'ordre dès que le service redémarre, sans aucune perte. Cela améliore à la fois la résilience, les performances et la maintenabilité du système.

Q6 : — Si on voulait ajouter une 3ème instance Node.js, quelle ligne faudrait-il modifier dans nginx.conf ?

Il suffit d'ajouter une ligne dans le bloc upstream backend du fichier nginx/nginx.conf :

```
upstream backend {
    server app1:3001;
    server app2:3002;
    server app3:3003;   ← ajouter cette ligne
}
```

Il faudrait également ajouter le service correspondant dans docker-compose.yml avec les variables d'environnement PORT: 3003 et INSTANCE: 'Instance-3', et créer un mapping de port 3003:3003.

8. CONCLUSION

Ce TP a permis de déployer avec succès une architecture distribuée complète, fonctionnelle et testée. L'ensemble des six composants — Nginx, Node.js (×2), MongoDB, Redis et RabbitMQ — ont été configurés et mis en interaction via Docker Compose, reproduisant fidèlement les conditions d'un système distribué en environnement de production.

Les principaux enseignements tirés de ce travail pratique sont les suivants :

- **Nginx assure efficacement la répartition de charge entre instances grâce à l'algorithme Round-robin**
- **MongoDB offre un stockage persistant et fiable, servant de source de vérité du système**
- **Redis améliore significativement les performances (gain de 6× mesuré) en réduisant les accès répétitifs à la base de données**
- **RabbitMQ renforce la résilience en découplant les traitements critiques des notifications asynchrones**
- **Docker Compose simplifie l'orchestration et garantit la reproductibilité de l'environnement**

La principale difficulté rencontrée — la gestion du timing de démarrage des conteneurs — illustre parfaitement un défi récurrent dans les architectures distribuées réelles : la nécessité d'implémenter des mécanismes de retry et de tolérance aux pannes dès la phase de conception. La solution apportée (boucle de retry avec délai progressif) est d'ailleurs une bonne pratique recommandée en production.

Ce TP constitue une excellente introduction aux architectures microservices modernes, où chaque composant est découplé, spécialisé et peut évoluer indépendamment des autres.